



React Learning & Revision Notes

This README contains quick revision notes for the major React concepts I have learned, with simple examples.

1. JSX

JSX allows us to write HTML-like syntax inside JavaScript.

```
function App() {  
  return (  
    <div>  
      <h1>Hello React</h1>  
    </div>  
  );  
}
```

JavaScript expressions can be used inside JSX using `{}`.

```
const name = "Gary";  
  
return <h1>Hello {name}</h1>;
```

2. Components

Components are reusable pieces of UI.

React component names should start with a capital letter.

```
function Welcome() {  
  return <h1>Welcome!</h1>;  
}  
  
function App() {  
  return (  
    <div>  
      <Welcome />  
    </div>  
  );  
}
```

3. Props

Props are used to pass data from a parent component to a child component.

Think:

Parent → Child

```
function User({ name, age }) {
  return (
    <div>
      <p>Name: {name}</p>
      <p>Age: {age}</p>
    </div>
  );
}

function App() {
  return <User name="Gary" age={25} />;
}
```

The parent sends:

```
<User name="Gary" age={25} />
```

The child receives:

```
function User({ name, age }) {
```

Props should be treated as read-only.

4. State - useState

State stores data belonging to a component that can change over time.

```
import { useState } from "react";

function Counter() {

  const [count, setCount] = useState(0);

  return (
    <div>
```

```
    <h2>{count}</h2>

    <button onClick={() => setCount(count + 1)}>
      Increment
    </button>

    <button onClick={() => setCount(count - 1)}>
      Decrement
    </button>

  </div>
);
}
```

Here:

```
const [count, setCount] = useState(0);
```

means:

```
count      → current state value
setCount   → function used to update the state
0          → initial value
```

Updating state causes React to re-render the component.

When new state depends on previous state, the safer form is:

```
setCount(prevCount => prevCount + 1);
```

5. Updating Arrays in State

Example:

```
const [counters, setCounters] = useState([
  { id: 1, value: 0 }
]);
```

Add a counter:

```
const addCounter = () => {  
  
  setCounters([  
    ...counters,  
    {  
      id: counters.length + 1,  
      value: 0  
    }  
  ]);  
  
};
```

Update one particular counter:

```
const incrementCounter = (id) => {  
  
  setCounters(  
    counters.map(counter =>  
      counter.id === id  
        ? { ...counter, value: counter.value + 1 }  
        : counter  
    )  
  );  
  
};
```

Meaning:

```
If ID matches  
  → copy counter  
  → increase value  
  
If ID doesn't match  
  → return counter unchanged
```

6. Event Handling

React events use camelCase.

```
<button onClick={handleClick}>  
  Click  
</button>
```

Example:

```
const handleClick = () => {  
  console.log("Clicked");  
};
```

We can also use an anonymous arrow function:

```
<button onClick={() => setCount(count + 1)}>  
  Increment  
</button>
```

Important:

```
onClick={() => setCount(count + 1)}
```

passes a function for React to execute when the click happens.

7. Forms

Example:

```
import { useState } from "react";  
  
function Form() {  
  const [name, setName] = useState("");  
  
  const handleSubmit = (event) => {  
    event.preventDefault();  
  
    console.log(name);  
  };  
  
  return (  
    <form onSubmit={handleSubmit}>  
  
      <input  
        value={name}  
        onChange={(event) => setName(event.target.value)}  
      />  
    </form>  
  );  
}
```

```
        <button type="submit">
          Submit
        </button>

      </form>
    );
  }
```

`event.target.value` gives the current input value.

`event.preventDefault()` prevents the browser's default form submission/reload.

8. useEffect

`useEffect` is used for side effects.

Examples:

- API calls
- timers
- event listeners
- syncing with browser APIs

Basic syntax:

```
useEffect(() => {

  // side effect

}, []);
```

useEffect with No Dependency Array

```
useEffect(() => {
  console.log("Runs after every render");
});
```

useEffect with Empty Dependency Array

```
useEffect(() => {
  console.log("Runs when component mounts");
});
```

```
}, []);
```

useEffect with Dependency

```
useEffect(() => {  
  console.log("Count changed");  
}, [count]);
```

Runs whenever `count` changes.

useEffect Cleanup

Example with timer:

```
useEffect(() => {  
  const timer = setInterval(() => {  
    console.log("Running...");  
  }, 1000);  
  
  return () => {  
    clearInterval(timer);  
  };  
}, []);
```

The returned function is the cleanup function.

It runs when the effect needs cleanup, including when the component unmounts.

9. useRef

`useRef` stores a value/reference that survives re-renders without causing a re-render when changed.

A common use is accessing DOM elements.

```
import { useRef } from "react";  
  
function InputFocus() {  
  const inputRef = useRef(null);
```

```
const focusInput = () => {
  inputRef.current.focus();
};

return (
  <div>

    <input ref={inputRef} />

    <button onClick={focusInput}>
      Focus Input
    </button>

  </div>
);
}
```

`inputRef.current` points to the actual input DOM element after it has been attached.

Another use is storing mutable values that don't need to appear on screen.

```
const renderCount = useRef(0);

renderCount.current++;
```

Changing `.current` does NOT trigger a re-render.

10. useContext

Context allows data to be shared with deeply nested components without manually passing props through every level.

This helps avoid prop drilling.

Create context:

```
import { createContext } from "react";

export const ThemeContext = createContext("light");
```

Provide the value:

```
<ThemeContext.Provider value="dark">
  <App />
```

```
</ThemeProvider>
```

Read the value:

```
import { useContext } from "react";
import { ThemeContext } from "../ThemeContext";

function Navbar() {

  const theme = useContext(ThemeContext);

  return <h2>Theme: {theme}</h2>;
}
```

Flow:

```
Provider
  ↓
Child
  ↓
Grandchild
  ↓
useContext()
```

The component using `useContext` gets the value from the nearest matching Provider above it.

11. API Calls in React - fetch()

APIs are commonly called inside `useEffect`.

```
import { useEffect, useState } from "react";

function Posts() {

  const [posts, setPosts] = useState([]);

  useEffect(() => {

    fetch("https://jsonplaceholder.typicode.com/posts")
      .then(response => response.json())
      .then(data => setPosts(data))
      .catch(error => console.error(error));

  }, []);
```

```
return (  
  <div>  
  
    {posts.map(post => (  
      <p key={post.id}>  
        {post.title}  
      </p>  
    ))}  
  
  </div>  
);  
}
```

Flow:

```
Component mounts  
↓  
useEffect runs  
↓  
fetch API  
↓  
response  
↓  
convert response to JSON  
↓  
setPosts(data)  
↓  
state changes  
↓  
component re-renders
```

12. API Calls with async/await

```
useEffect(() => {  
  
  const fetchPosts = async () => {  
  
    try {  
  
      const response = await fetch(  
        "https://jsonplaceholder.typicode.com/posts"  
      );  
  
      const data = await response.json();  
  
      setPosts(data);  
    }  
  }  
});
```

```
    } catch (error) {  
  
        console.error(error);  
  
    }  
  
};  
  
fetchPosts();  
  
}, []);
```

13. Loading and Error States

Real API calls should usually track loading and errors.

```
const [posts, setPosts] = useState([]);  
const [loading, setLoading] = useState(false);  
const [error, setError] = useState(null);
```

Example:

```
const fetchPosts = async () => {  
  
    setLoading(true);  
  
    try {  
  
        const response = await fetch(url);  
  
        const data = await response.json();  
  
        setPosts(data);  
  
    } catch (error) {  
  
        setError(error.message);  
  
    } finally {  
  
        setLoading(false);  
  
    }  
  
};
```

Then:

```
if (loading) {  
  return <p>Loading...</p>;  
}  
  
if (error) {  
  return <p>Error: {error}</p>;  
}
```

14. Axios

Axios is another library used for HTTP/API requests.

Install:

```
npm install axios
```

Import:

```
import axios from "axios";
```

GET request:

```
const fetchPosts = async () => {  
  try {  
    const response = await axios.get(  
      "https://jsonplaceholder.typicode.com/posts"  
    );  
    setPosts(response.data);  
  } catch (error) {  
    console.error(error);  
  }  
};
```

Important difference:

With `fetch`:

```
const response = await fetch(url);
const data = await response.json();
```

With Axios:

```
const response = await axios.get(url);
const data = response.data;
```

Axios automatically parses JSON responses in normal cases.

15. Axios POST Request

```
const createPost = async () => {

  const newPost = {
    title: "React",
    body: "Learning Axios"
  };

  const response = await axios.post(
    "https://jsonplaceholder.typicode.com/posts",
    newPost
  );

  console.log(response.data);

};
```

16. Axios Promise Syntax

Axios can also be written using `.then()`:

```
axios
  .get(url)
  .then(response => {
    setPosts(response.data);
  })
  .catch(error => {
    setError(error.message);
  });
```

```
  })  
  .finally(() => {  
    setLoading(false);  
  });
```

Correct:

```
.finally(() => setLoading(false))
```

Not:

```
.finally(setLoading(false))
```

because the second version executes the function immediately.

17. React Router

Install:

```
npm install react-router-dom
```

Basic routing:

```
import {  
  BrowserRouter,  
  Routes,  
  Route,  
  Link  
} from "react-router-dom";  
  
function App() {  
  return (  
    <BrowserRouter>  
  
      <nav>  
        <Link to="/">Home</Link>  
        <Link to="/about">About</Link>  
      </nav>  
  
      <Routes>  
  
        <Route
```

```
        path="/"
        element={<Home />}
      />

      <Route
        path="/about"
        element={<About />}
      />

    </Routes>

  </BrowserRouter>
);
}
```

[Link](#) changes routes without performing a traditional full-page browser reload.

18. Query Parameters

Example URL:

```
/about?topic=team
```

Read it:

```
const [searchParams, setSearchParams] = useSearchParams();

const topic =
  searchParams.get("topic") || "general";
```

Change it:

```
setSearchParams({
  topic: "Team"
});
```

URL becomes:

```
/about?topic=Team
```

19. Nested Routes and Outlet

Routes can contain child routes.

```
<Route path="/about" element={<About />}>

  <Route
    path="team"
    element={<Team />}
  />

</Route>
```

Inside `About`:

```
import { Outlet } from "react-router-dom";

function About() {

  return (
    <div>

      <h1>About</h1>

      <Outlet />

    </div>
  );
}
```

For:

```
/about/team
```

React renders:

```
About
  ↓
Outlet
  ↓
Team
```

`Outlet` is the location where the nested route component appears.

20. Redux

Redux is used for managing shared/global application state.

Basic Redux concepts:

```
Store
Action
Reducer
Dispatch
```

Flow:

```
Component
↓
dispatch(action)
↓
Reducer receives current state + action
↓
Reducer returns new state
↓
Store saves new state
↓
React components using that state update
```

21. Redux Action

Example:

```
export const increment = () => ({
  type: "INCREMENT"
});

export const decrement = () => ({
  type: "DECREMENT"
});
```

Calling:

```
increment()
```

returns:

```
{
  type: "INCREMENT"
}
```

Actions describe what happened.

22. Redux Reducer

```
const initialState = {
  count: 0
};

const counterReducer = (
  state = initialState,
  action
) => {

  switch (action.type) {

    case "INCREMENT":

      return {
        count: state.count + 1
      };

    case "DECREMENT":

      return {
        count: state.count - 1
      };

    default:
      return state;

  }

};

export default counterReducer;
```

Important:

```
state = initialState
```

does NOT reset the state every time.

`initialState` is used when `state` is `undefined`, such as during initialization.

Afterwards Redux passes the current state to the reducer.

23. Redux Store

Modern Redux applications normally use Redux Toolkit.

Install:

```
npm install @reduxjs/toolkit react-redux
```

Example:

```
import { configureStore } from "@reduxjs/toolkit";
import counterReducer from "./counterReducer";

export const store = configureStore({
  reducer: {
    counter: counterReducer
  }
});
```

Then provide the store:

```
import { Provider } from "react-redux";

<Provider store={store}>
  <App />
</Provider>
```

24. useSelector

`useSelector` reads data from Redux state.

If the store structure is:

```
{
  counter: {
    count: 10
  }
}
```

then:

```
const count = useSelector(
  state => state.counter.count
);
```

gives:

```
10
```

25. useDispatch

`useDispatch` gives the Redux dispatch function.

```
const dispatch = useDispatch();
```

Then:

```
dispatch(increment());
```

Flow:

```
increment()
↓
{ type: "INCREMENT" }
↓
dispatch
↓
reducer
↓
new state
```

26. Async Redux / Thunk

Normal Redux actions look like:

```
{
  type: "FETCH_POSTS_SUCCESS",
  payload: data
}
```

For asynchronous operations such as API calls, thunk middleware allows an action creator to return a function.

Example:

```
export const fetchPosts = () => async (dispatch) => {

  dispatch(fetchPostRequest());

  try {

    const response = await fetch(
      "https://jsonplaceholder.typicode.com/posts"
    );

    const data = await response.json();

    dispatch(fetchPostSuccess(data));

  } catch (error) {

    dispatch(
      fetchPostFailure(error.message)
    );

  }

};
```

The important syntax:

```
() => async (dispatch) => {}
```

means:

```
fetchPosts()
↓
```

```
returns an async function
↓
Redux thunk receives that function
↓
Thunk supplies dispatch
↓
async function runs
```

This is why we can write:

```
dispatch(fetchPosts());
```

without manually passing `dispatch` into `fetchPosts`.

27. Redux Async Actions

Request:

```
export const fetchPostRequest = () => ({
  type: "FETCH_POSTS_REQUEST"
});
```

Success:

```
export const fetchPostSuccess = posts => ({
  type: "FETCH_POSTS_SUCCESS",
  payload: posts
});
```

Failure:

```
export const fetchPostFailure = error => ({
  type: "FETCH_POSTS_FAILURE",
  payload: error
});
```

`payload` carries data associated with the action.

Example:

```
{
  type: "FETCH_POSTS_SUCCESS",
  payload: [
    { id: 1, title: "Hello" }
  ]
}
```

28. Async Redux Reducer

```
const initialState = {

  posts: [],
  loading: false,
  error: null

};

const postReducer = (
  state = initialState,
  action
) => {

  switch (action.type) {

    case "FETCH_POSTS_REQUEST":

      return {
        ...state,
        loading: true,
        error: null
      };

    case "FETCH_POSTS_SUCCESS":

      return {
        ...state,
        loading: false,
        posts: action.payload
      };

    case "FETCH_POSTS_FAILURE":

      return {
        ...state,
        loading: false,
        error: action.payload
      };

  }

}
```

```
    default:
      return state;
  }
};
```

The three states of an API request are basically:

```
REQUEST
↓
loading = true

SUCCESS
↓
loading = false
data = payload

FAILURE
↓
loading = false
error = payload
```

29. Redux preloadedState

Redux Toolkit's `configureStore` can optionally receive initial/preloaded data:

```
configureStore({
  reducer: {
    posts: postReducer
  },
  preloadedState: {
    posts: {
      posts: [],
      loading: false,
      error: null
    }
  }
});
```

But `preloadedState` is NOT required if the reducer already has an initial state.

It is useful when initial data comes from somewhere else, such as:

- localStorage
- server-rendered data
- persisted application state

30. Tailwind CSS

Tailwind provides utility classes directly inside JSX.

Example:

```
<button
  className="
    bg-blue-500
    text-white
    p-3
    rounded-lg
  "
>
  Click Me
</button>
```

Instead of writing CSS such as:

```
button {
  background-color: blue;
  color: white;
  padding: 12px;
  border-radius: 8px;
}
```

Tailwind provides utilities.

31. Common Tailwind Classes

Spacing:

```
p-2      → padding
p-4
px-4     → horizontal padding
py-2     → vertical padding
```

```
m-2      → margin  
mt-4     → margin top  
mb-4     → margin bottom
```

Text:

```
text-sm  
text-lg  
text-xl  
text-2xl  
text-3xl  
  
font-bold  
font-semibold  
  
text-white  
text-blue-500
```

Background:

```
bg-blue-500  
bg-red-500  
bg-gray-900
```

Border:

```
border  
border-2  
border-black  
border-blue-500
```

Rounded corners:

```
rounded  
rounded-md  
rounded-lg  
rounded-xl  
rounded-2xl  
rounded-full
```

Layout:

```
flex
items-center
justify-center
justify-between
justify-around

grid
grid-cols-2
grid-cols-3
```

Width/height:

```
w-full
h-full
min-h-screen
```

32. Tailwind Hover

```
<button
  className="
    bg-blue-500
    hover:bg-blue-700
    text-white
    p-3
    rounded-lg
  "
>
  Hover Me
</button>
```

Scale on hover:

```
<button
  className="
    inline-block
    hover:scale-110
    transition-transform
    duration-300
  "
>
  Hover
</button>
```

33. Tailwind Dark Mode

Example:

```
<div
  className="
    bg-white
    text-black
    dark:bg-gray-900
    dark:text-white
    min-h-screen
  "
>
  Hello
</div>
```

If the `dark` class is placed on the root HTML element:

```
<html class="dark">
```

then:

```
dark:bg-gray-900
dark:text-white
```

become active.

React example:

```
const [isDarkMode, setIsDarkMode] = useState(false);

useEffect(() => {

  if (isDarkMode) {

    document.documentElement
      .classList.add("dark");

  } else {

    document.documentElement
      .classList.remove("dark");

  }

}
```

```
}, [isDarkMode]));
```

`document.documentElement` refers to the `<html>` element.

34. localStorage

`localStorage` stores data in the browser and survives page reloads and browser restarts until removed/cleared.

Save:

```
localStorage.setItem(  
  "theme",  
  "dark"  
);
```

Read:

```
const theme =  
  localStorage.getItem("theme");
```

Remove:

```
localStorage.removeItem("theme");
```

Example with dark mode:

```
const [isDarkMode, setIsDarkMode] = useState(  
  () => localStorage.getItem("theme") === "dark"  
);
```

35. React Inline CSS

React inline styles use JavaScript objects.

```
<button  
  style={{
```

```
    margin: "10px",
    padding: "10px",
    borderRadius: "10px",
    border: "none"
  }}
>
  Click
</button>
```

Why two braces?

```
style={{ margin: "10px" }}
```

Outer `{}`:

```
Enter JavaScript from JSX
```

Inner `{}`:

```
JavaScript object
```

Equivalent:

```
const buttonStyle = {
  margin: "10px",
  padding: "10px"
};

<button style={buttonStyle}>
  Click
</button>
```

React inline CSS uses camelCase:

CSS	React
background-color	→ backgroundColor
border-radius	→ borderRadius
font-size	→ fontSize

36. Props vs State vs Context vs Redux

A useful mental model:

```
PROPS
Parent → Child

STATE
Component's changing data

CONTEXT
Share data across a component tree

REDUX
Centralized/shared application state
```

Example:

```
Small local counter
→ useState

Parent sending username to child
→ props

Theme used throughout component tree
→ Context

Large app with shared user/cart/API state
→ Redux may be useful
```

37. useState vs useRef

Both can preserve values across renders, but they behave differently.

```
useState
→ stores data
→ updating it causes re-render

useRef
→ stores mutable value/reference
→ changing .current does NOT cause re-render
```

Example:

```
const [count, setCount] = useState(0);  
  
const countRef = useRef(0);
```

Changing:

```
setCount(10);
```

causes render.

Changing:

```
countRef.current = 10;
```

does not itself cause render.

38. Quick React Mental Model

A React component is basically:

```
Props come in  
↓  
Component executes  
↓  
State/hooks provide data  
↓  
JSX is returned  
↓  
React renders UI  
↓  
User interacts  
↓  
State changes  
↓  
Component executes again  
↓  
UI updates
```

39. Important Hooks Rule

Hooks should be called at the top level of React components/custom hooks.

Good:

```
function App() {  
  
  const [count, setCount] = useState(0);  
  
  return <div>{count}</div>;  
}
```

Avoid calling hooks conditionally:

```
if (something) {  
  const [count, setCount] = useState(0); // ❌  
}
```

React relies on hooks being called consistently.

40. Final Revision Cheat Sheet

```
useState  
→ local changing state  
  
useEffect  
→ side effects  
  
useRef  
→ DOM reference / persistent mutable value without re-render  
  
useContext  
→ consume shared Context value  
  
Props  
→ parent passes data to child  
  
fetch / Axios  
→ API communication  
  
React Router  
→ client-side routing  
  
Redux  
→ centralized/shared state management  
  
useSelector
```

→ read Redux state

useDispatch

→ dispatch Redux actions

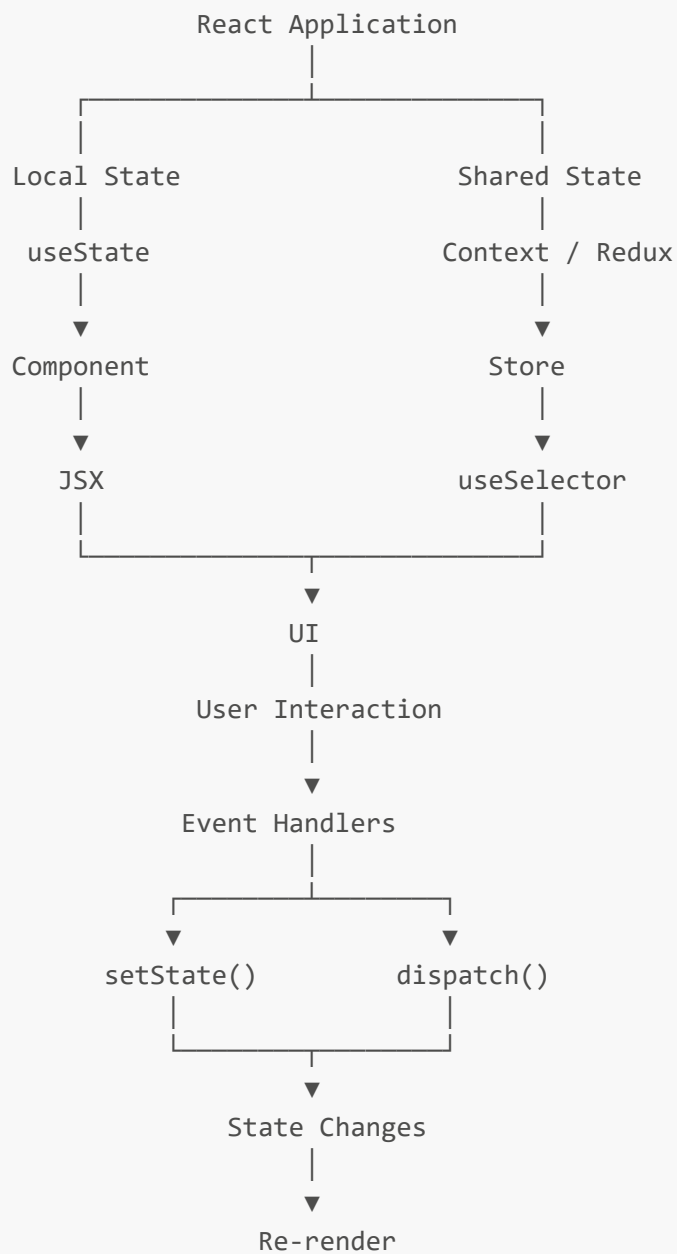
Redux Thunk

→ async Redux logic

Tailwind

→ utility-first CSS classes

React Data Flow Summary



Core Rule to Remember

React is mainly about:

Data → UI → User Interaction → State Change → Re-render

Once this flow is understood, most React concepts become much easier to connect.